UNIT 1

1.1 INTRODUCTION

A database is a collection of related data. By data, we mean known facts that can be recorded and that have implicit meaning. For example, consider the names, telephone numbers, and addresses of the people you know. Nowadays, this data is typically stored in mobile phones, which have their own simple database software. This data can also be recorded in an indexed address book or stored on a hard drive, using a personal computer and software such as Microsoft Access or Excel. This collection of related data with an implicit meaning is a database.

Data: Data is raw fact or figures or entity.

Information: The processed data is called information.

Database: A database is a collection of related data.

For example, a university database might contain information about the following:

- Entities such as students, faculty and courses.
- Relationships between entities, such as students' enrollment in courses, faculty teaching courses.

Database Management System: A Database Management System (DBMS) is a collection of program that enables user to create, maintain and manipulate a database.

The DBMS is hence a general purpose software system that facilitates the process of defining, constructing and manipulating database for various applications

1.2 Characteristics of the Database Approach

- **Self-Describing Nature of a Database System-** in the database approach, the data is defined by the users and this defined data describes itself. Now, what do I mean by defining itself? It means that the metadata or data about data is also stored in the database, specifically in the DBMS catalog.Now, what is this metadata? Metadata means data about data, like the constraints in the database, the structure of the database, the columns of the database, etc.
- **Insulation between Programs and Data, and Data Abstraction-** The DBMS or Database Management System provides the conceptual view of the data that is easily readable to the user and hides many details of the data such as how the data is stored or the constraints used on the data. This conceptual view is known

as the data model. Hence the data model hides the implementation and storage details of the data thus creating insulation between the program and the data.

- **Support of Multiple Views of the Data-** In the Database Approach, a single database is implemented and multiple users access this database.
- **Sharing of Knowledge and Multi-user Transaction Processing-** Sharing of knowledge means that multiple users are accessing common data like a dean and the head of the department are accessing the STUDENTS database. Hence both users are sharing the student records.

A transaction is a process that is accessing one or more databases by reading, writing, or deleting the data records.

Multi-user Transaction Processing as the name implies that the database must allow multiple users to process their respective transactions

1.4 Actors on the Scene

There are two categories of people behind DBMS.

a) Those who actually use and control the database content, and those who design, develop and maintain database applications (called ―Actors on the Scene)
b) Those who design and develop the DBMS software and related tools, and the computer systems operators (called ―Workers Behind the Scene).

a)Actors on the Scene

1. Database Administrator (DBA): DBA is a person who is responsible for authorizing access to the database, coordinating and monitoring its use, and acquiring software and hardware resources as needed.

2. Database Designers: They are responsible for identifying the data to be stored and for choosing an appropriate way to organize it. They also define views for different categories of users. The final design must be able to support the requirements of all the user sub-groups.

3. End Users: These are persons who access the database for querying, updating, and report generation. They are main reason for database's existence.

- **Casual end users**: use database occasionally, needing different information each time; use query language to specify their requests; typically middle- or highlevel managers.
- **Naive/Parametric end users:** Typically the biggest group of users; frequently query/update the database using standard canned transactions that have been carefully programmed and tested in advance. Examples:

  - bank tellers check account balances, post withdrawals/deposits

- reservation clerks for airlines, hotels, etc., check availability of seats/rooms and make reservations

- **Sophisticated end users:** engineers, scientists, business analysts who implement their own applications to meet their complex needs.
- **Stand-alone users:** Use "personal" databases, possibly employing a specialpurpose (e.g., financial) software package. Mostly maintain personal databases using ready-to-use packaged applications.

- An example is a tax program user that creates its own internal database.
- Another example is maintaining an address book

4. System Analysts, Application Programmers, Software Engineers:

- **System Analysts:** determine needs of end users, especially naive and parametric users, and develop specifications for canned transactions that meet these needs.
- **Application Programmers**: Implement, test, document, and maintain programs that satisfy the specifications mentioned above.

b) Workers Behind the Scene

1)DBMS system designers/implementors: provide the DBMS software that is at the foundation of all this.

2) Tool developers: design and implement software tools facilitating database system design, performance monitoring, creation of graphical user interfaces, prototyping, etc.

3) Operators and maintenance personnel: responsible for the day-to-day operation of the system.

1.6 Advantages of Using the DBMS Approach

Controlling Redundancy - In DBMS, each data item is stored in such a way so that there is no inconsistency in data, and moreover, it saves a lot of storage space!

Restricting unauthorized access

DBMS provides security and authorization so that some users are not authorized to access all data in the database

Providing persistent storage
A DBMS provides persistent storage for program objects which led to the emergence of Object-Oriented database management system. Hence, a complex object can be stored permanently in an O.O. DBMS. **Such an object is said to be persistent as it can then be directly retrieved by another program**.
Provides multiple user interfaces
There are different types of users with different technical knowledge. So, a **DBMS provides different user interfaces**.
Providing backup and recovery
A DBMS facilitates backup and recovery in the event of a software failure, which makes **the chances of data loss almost negligible!**
Representing complex relationships among data
A DBMS has the **capability to represent a variety of complex relationships among data, as well as to retrieve and update related data efficiently.** A good example of that is the use of foreign keys!
Enforcing integrity constraints
A DBMS **provides capabilities for defining and enforcing integrity constraints** that must hold for data. Example: specifying the data type for each data item.
Permitting inferencing and actions
Some database systems provide capabilities for defining deduction rules for inferencing new information from stored database facts (Database rules are deduced from facts!)
Atomicity in data
In a DBMS, **every transaction is 'atomic'**.This means that the transaction should either be fully complete or must not start at all.

2.1 Data Models, Schemas, and Instances

Data abstraction generally refers to the suppression of details of data organization and storage, and the highlighting of the essential features for an improved understanding of data.

A data model—a collection of concepts that can be used to describe the structure of a database—provides the necessary means to achieve this abstraction.

2.1.1 Categories of Data Models

- **High-level or conceptual data models** provide concepts that are close to the way many users perceive data. Conceptual data models use concepts such as entities, attributes, and relationships. An **entity** represents a real-world object or concept, such as an employee or a project from the mini world that is described in the database. An **attribute** represents some property of interest that further describes an entity, such as the employee's name or salary. A **relationship** among two or more entities represents an association among the

entities, for example, a works-on relationship between an employee and a project

- **Low-level or physical data models** provide concepts that describe the details of how data is stored on the computer storage media. Physical data models describe how data is stored as files in the computer by representing information such as record formats, record orderings, and access paths.
- **Representational (or implementation) data models** which provide concepts that may be easily understood by end users but that are not too far removed from the way data is organized in computer storage.

These include relational data base, h/w model and hierarchical data model
These data models are the models used most frequently in traditional commercial DBMSs.
Representational data models represent data by using record structures and hence are sometimes called **recordbased data models**

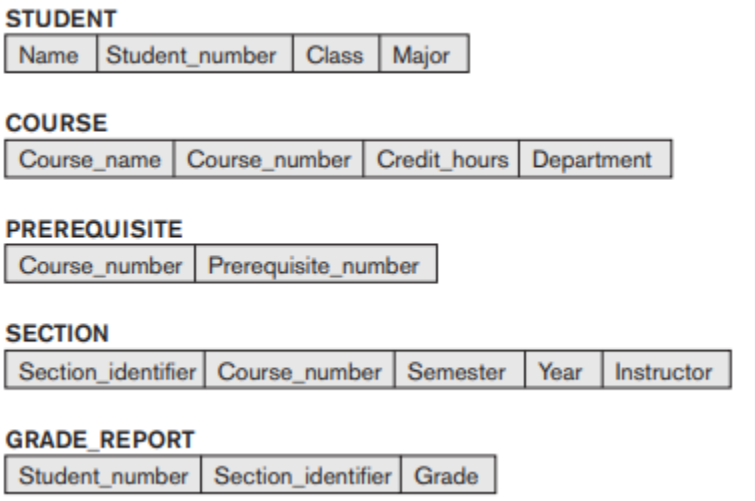
2.1.2 Schemas, Instances and Database State

Schemas

In a data model, it is important to distinguish between the description of the database and the database itself. The description of a database is called the **database schema**

A displayed schema is called a **schema diagram**

Example:

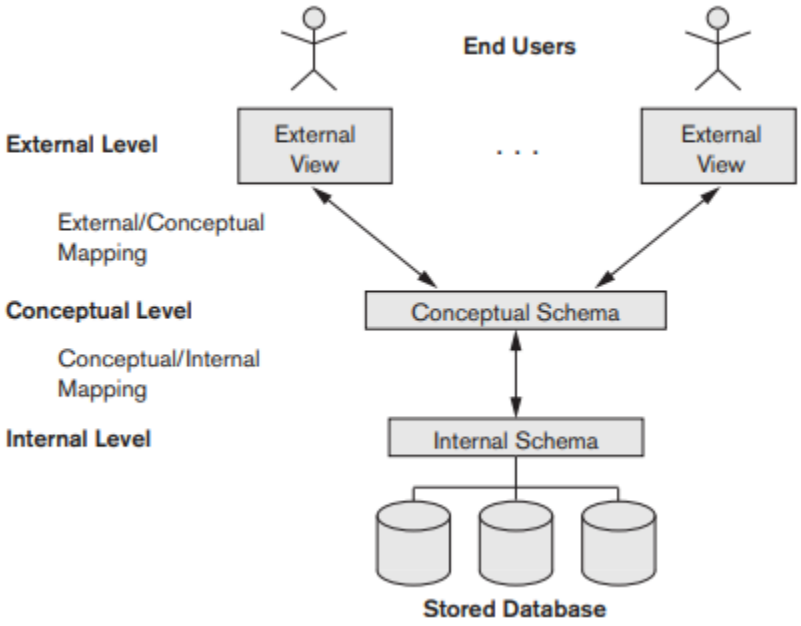


Instances and Database State

The data in the database at a particular moment in time is called a **database state** or **snapshot**. It is also called the current set of occurrences or **instances** in the database. In a given database state, each schema construct has its own current set of instances; **For example**, the STUDENT construct will contain the set of individual student entities (records) as its instances.

2.2 Three-Schema Architecture and Data Independence

Figure 2.2
The three-schema architecture.



The goal of the three-schema architecture, illustrated in Figure 2.2, is to separate the user applications from the physical database. In this architecture, schemas can be defined at the following three levels:

1. **The internal level** has an internal schema, which **describes the physical storage** structure of the database. The internal schema uses a physical data model and describes the complete details of data storage and access paths for the database.
2. **The conceptual level** has a conceptual schema, which **describes the structure** of the whole database for a community of users and t hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints. Usually, a representational data

model is used to describe the conceptual schema when a database system is implemented. This implementation conceptual schema is often based on a conceptual schema design in a high-level data model.

3. The **external or view level** includes a number of external schemas or user views. Each external schema **describes the part of the database that a particular user group is interested** in and hides the rest of the database from that user group.

2.2.2 Data Independence

Data independence can be defined as the capacity to change the schema at one level without changing the schema at next higher level. We can define two types of data independence:

1. **Logical data independence** is the capacity to change the conceptual schema without having to change external schemas or application programs. We may change the conceptual schema to expand the database (by adding a record type or data item), to change constraints, or to reduce the database (by removing a record type or data item). In the last case, external schemas that refer only to the remaining data should not be affected.
2. **Physical data independence** is the capacity to change the internal schema without having to change the conceptual schema. Hence, the external schemas need not be changed as well. Using this property, we can change the storage device of the database without affecting the logical schema.

2.3 Database Languages and Interfaces

The DBMS must provide appropriate languages and interfaces for each category of users. In this section we discuss the types of languages and interfaces provided by a DBMS and the user categories targeted by each interface

2.3.1 DBMS Languages

1. Data definition language (DDL)

Data definition language (DDL) creates the framework of the database by specifying the database schema, which is the structure that represents the organization of data. Its common uses include the creation and alteration of tables, files, indexes and columns within the database. This language also allows users to rename or drop the existing database or its components. Here's a list of DDL statements:

- **CREATE:** Creates a new database or object, such as a table, index or column
- **ALTER:** Changes the structure of the database or object
- **DROP:** Deletes the database or existing objects
- **RENAME:** Renames the database or existing objects

2. Data manipulation language (DML)

Data manipulation language (DML) provides operations that handle user requests, offering a way to access and manipulate the data that users store within a database. Its common functions include inserting, updating and retrieving data from the database. Here's a list of DML statements:

- **INSERT:** Adds new data to the existing database table
- **UPDATE:** Changes or updates values in the table
- **DELETE:** Removes records or rows from the table
- **SELECT:** Retrieves data from the table or multiple tables

There are basically two types:

• **Procedural DMLs** require a user to specify what data are needed and how to

get those data, The DML component of the PL/SQL language is procedural.

•**Nonprocedural DMLs** require a user to specify what data are needed without

specifying how to get those data. The DML component of the SQL language is

nonprocedural

3. The storage definition language (SDL)

The storage definition language (SDL), is used to specify the internal schema. The mappings between the two schemas may be specified in either one of these languages.

4. The view definition language (VDL)

the view definition language (VDL), to specify user views and their mappings to the conceptual schema, but in most DBMSs the DDL is used to define both conceptual and external schemas. In relational DBMSs, SQL is used in the role of VDL to define user or application views as results of predefined queries.

2.3.2 DBMS Interfaces

User-friendly interfaces provided by a DBMS may include the following:

- **Menu-based Interfaces for Web Clients or Browsing**. These interfaces present the user with lists of options (called menus) that lead the user through the formulation of a request. Menus do away with the need to memorize the specific commands and syntax of a query language; rather, the query is composed step-bystep by picking options from a menu that is displayed by the system. Pull-down menus are a very popular technique in Web-based user interfaces. They are also often used in browsing interfaces, which allow a user to look through the contents of a database in an exploratory and unstructured manner.
- **Apps for Mobile Devices.** These interfaces present mobile users with access to their data. For example, banking. The apps have built-in programmed interfaces that allow users to login using their account name and password; the apps then provide a limited menu of options for mobile access to the user data.
- **Forms-based Interfaces**. A forms-based interface displays a form to each user. Users can fill out all of the form entries to insert new data, or they can fill out only certain entries, in which case the DBMS will retrieve matching data for the remaining entries.
- **Graphical User Interfaces**. A GUI typically displays a schema to the user in diagrammatic form. The user then can specify a query by manipulating the diagram. In many cases, GUIs utilize both menus and forms.

- **Natural Language Interfaces**. These interfaces accept requests written in English or some other language and attempt to understand them. A natural language interface usually has its own schema, which is similar to the database conceptual schema, as well as a dictionary of important words.
- **Keyword-based Database Search.** These are somewhat similar to Web search engines, which accept strings of natural language (like English or Spanish) words and match them with documents at specific sites (for local search engines) or Web pages on the Web at large (for engines like Google or Ask).
- **Speech Input and Output.** Limited use of speech as an input query and speech as an answer to a question or result of a request is becoming commonplace. Applications with limited vocabularies, such as inquiries for telephone directory, flight arrival/departure, and credit card account information, are allowing speech for input and output to enable customers to access this information
- **Interfaces for Parametric Users**. Parametric users, such as bank tellers, often have a small set of operations that they must perform repeatedly. For example, a teller is able to use single function keys to invoke routine and repetitive transactions such as account deposits or withdrawals, or balance inquiries. Systems analysts and programmers design and implement a special interface for each known class of naive users. Usually a small set of abbreviated commands is included, with the goal of minimizing the number of keystrokes required for each request.
- **Interfaces for the DBA**. Most database systems contain privileged commands that can be used only by the DBA staff. These include commands for creating accounts, setting system parameters, granting account authorization, changing a schema, and reorganizing the storage structures of a database.

2.4 The Database System Environment

A DBMS is a complex software system. In this section we discuss the types of software components that constitute a DBMS and the types of computer system software with which the DBMS interacts.

2.4.1 DBMS Component Modules

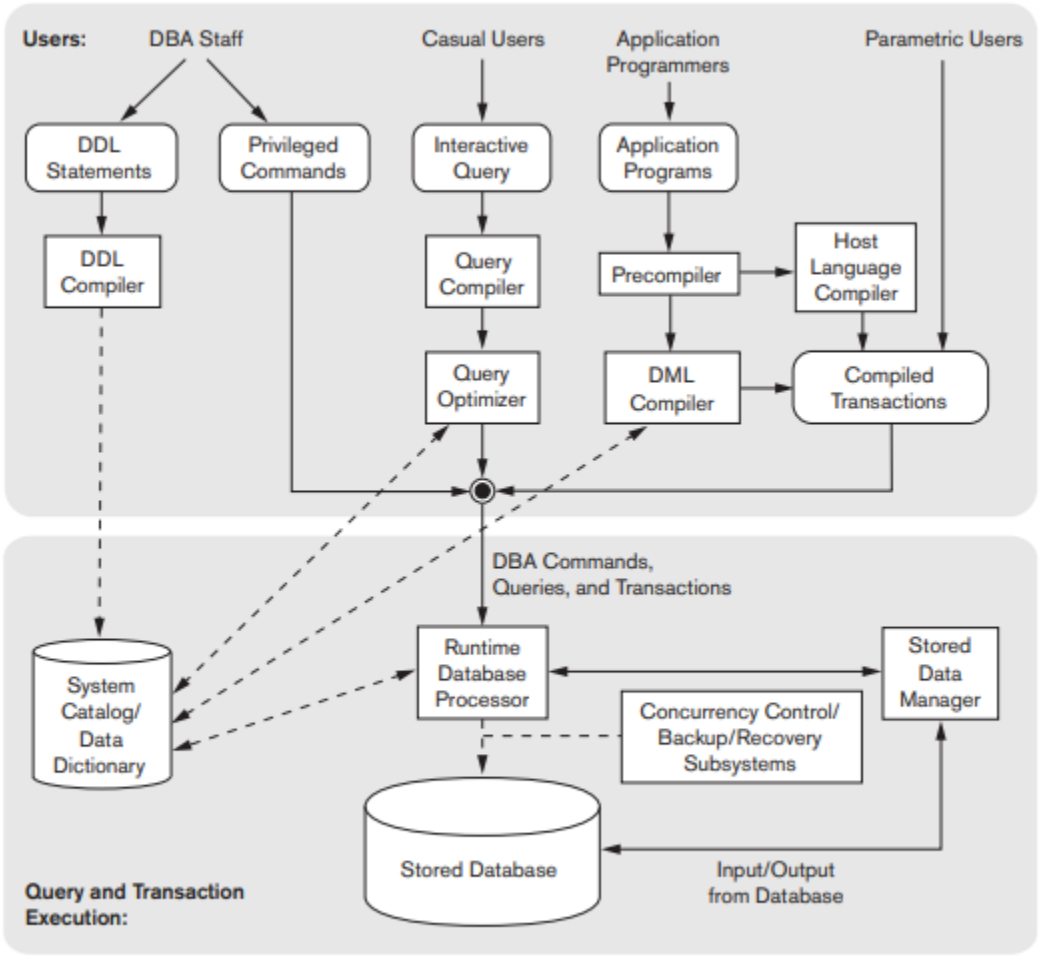


Figure 2.3
Component modules of a DBMS and their interactions.

Figure 2.3 illustrates, in a simplified form, the typical DBMS components. The figure is divided into two parts.

- The top part of the figure refers to the various users of the database environment and their interfaces.
- The lower part shows the internal modules of the DBMS responsible for storage of data and processing of transactions.

The database and the DBMS catalog are usually stored on disk. Access to the disk is controlled primarily by the operating system (OS), which schedules disk read/write. Many DBMSs have their own buffer management module to schedule disk read/write, because management of buffer storage has a considerable effect on performance.

Reducing disk read/write improves performance considerably. A higher-level stored data manager module of the DBMS controls access to DBMS information that is stored on disk, whether it is part of the database or the catalog.

the top part of Figure, It shows interfaces for the **DBA staff**, casual users who work with interactive interfaces to formulate queries, **application programmers** who create programs using some host programming languages, and **parametric users** who do data entry work by supplying parameters to predefined transactions. The DBA staff works on defining the database and tuning it by making changes to its definition using the DDL and other privileged commands. The DDL compiler processes schema definitions, specified in the DDL, and stores descriptions of the schemas (meta-data) in the DBMS catalog. The catalog includes information such as the names and sizes of files, names and data types of data items, storage details of each file, mapping information among schemas, and constraints. Casual users and persons with occasional need for information from the database interact using some form of interface, which we call the interactive query interface. These queries are parsed and validated for correctness of the query syntax, the names of files and a query compiler that compiles them into an internal form. This internal query is subjected to query optimization, the query optimizer is concerned with the rearrangement and possible reordering of operations, elimination of redundancies, and use of correct algorithms and indexes during execution. The **precompiler** extracts DML commands from an application program written in a host programming language. These commands are sent to the DML compiler for compilation into object code for database access. The rest of the program is sent to the host language compiler. The object codes for the DML commands and the rest of the program are linked, forming a canned transaction whose executable code includes calls to the runtime database processor. It is now common to have the client program that accesses the DBMS running on a separate computer from the computer on which the database resides. The former is called the client computer running a DBMS client software and the latter is called the **database server**. In some cases, the client accesses a middle computer, called the **application server**, which in turn accesses the database server

2.4.2 Database System Utilities

DBMSs have database utilities that help the DBA manage the database system. Common utilities have the following types of functions:

■ **Loading**. A loading utility is used to load existing data files—such as text files or sequential files—into the database. Usually, the current (source) format of the data file and the desired (target) database file structure are specified to the utility, which then automatically reformats the data and stores it in the database.

■ **Backup**. A backup utility creates a backup copy of the database, usually by dumping the entire database onto tape or other mass storage medium. The backup copy can be used to restore the database in case of catastrophic disk failure.

■ **Database storage reorganization**. This utility can be used to reorganize a set of database files into different file organizations and create new access paths to improve performance.

■ **Performance monitoring**. Such a utility monitors database usage and provides statistics to the DBA. The DBA uses the statistics in making decisions such as whether or not to reorganize files or whether to add or drop indexes to improve performance.

2.4.3 Tools, Application Environments, and Communications Facilities

Tool that can be quite useful in large organizations is an expanded data dictionary (or data repository) system. In addition to storing catalog information about schemas and constraints, the data dictionary stores other information, such as design decisions, usage standards, application program descriptions, and user information. Such a system is also called an information repository.

Application development environments, such as PowerBuilder (Sybase) or JBuilder (Borland), have been quite popular. These systems provide an environment for developing database applications

The DBMS also needs to interface with **communications software**, whose function is to allow users at locations remote from the database system site to access the database through computer terminals, workstations, or personal computers.

2.5 Centralized and Client/Server Architectures for DBMSs

2.5.1 Centralized DBMSs Architecture

In Centralized Architecture, the mainframe computers are used for processing all system functions including User application Programs and User Interface Programs as well as DBMS functionalities. This is because in earlier days, most users accessed such systems via Computer Terminals, which can't Process and they have only display capability. Therefore the Processing used to take place in these Computer Systems and the display information is sent to display terminals and these terminals are connected to mainframe computers via various kinds of Networks.

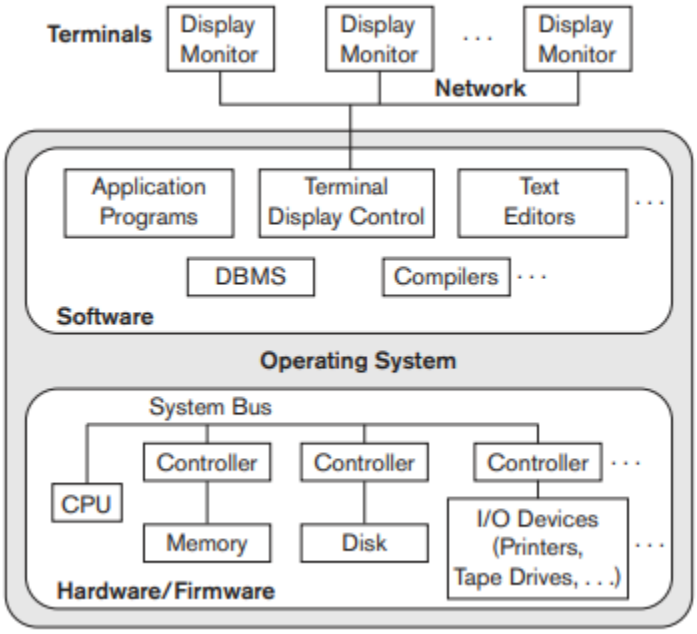


Figure 2.4
A physical centralized architecture.

2.5.2 Basic Client/Server Architectures

In client/server architecture a large number of PCs, workstations, file servers, printers, database servers, Web servers, e-mail servers, and other software and equipment are connected via a network. . For example, it is possible to connect a number of PCs or small workstations as clients to a file server that maintains the files of the client machines. Another machine can be designated as a printer server by being connected to various printers; all print requests by the clients are forwarded to this machine. Web servers or e-mail servers also fall into the specialized server category. The resources provided by specialized servers can be accessed by many client machines. The client

machines provide the user with the appropriate interfaces to utilize these servers, as well as with local processing power to run local applications.

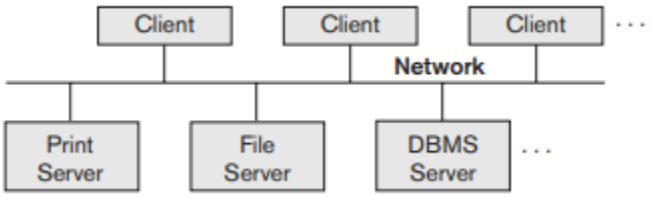


Figure 2.5
Logical two-tier client/server architecture.

Two main types of basic DBMS architectures were created on this underlying client/server framework: two-tier and three-tier

2.5.3 Two-Tier Client/Server Architectures for DBMSs

Here Two-tier means that our Architecture has two layers, which are client layer and Datalayer. In Client layer we have several Client machines which can have the access to the database server. The API present on the client machine will establish the connection between the machine and the Database server through JDBC somthing else.This is because Clients and Database Server may be at different different locations.Once this connection gets established,the Interface present on the client machine contains an Application Program on the back-side which contains a query. This query will be processed by the Database server and in turn the queried information will be sent to the client machine.

For example if we query the database to retrieve some information, the query will be Processed by Database server and that information will be sent to the client by Database server itself!!!

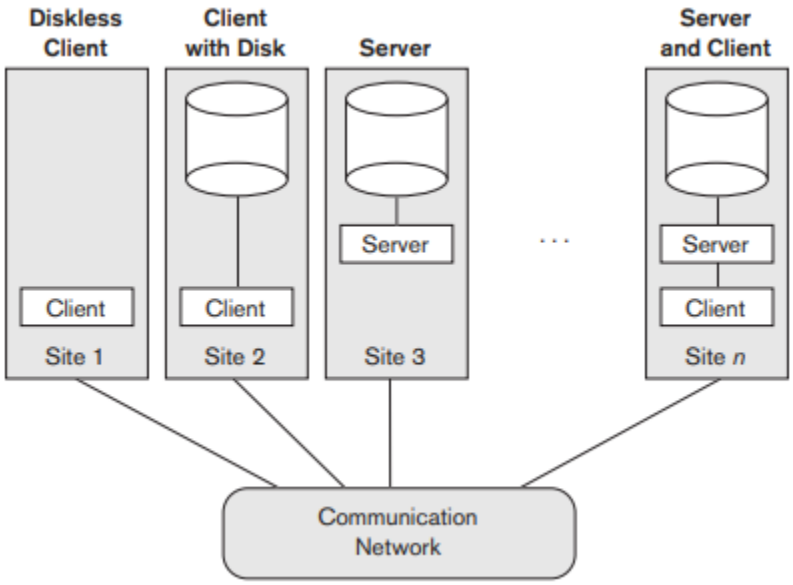


Figure 2.6
Physical two-tier client/server architecture.

2.5.4 Three-Tier and n-Tier Architectures for Web Applications

Many Web applications use an architecture called the three-tier architecture, which adds an intermediate layer between the client and the database server, as illustrated in Figure 2.7(a). This intermediate layer or middle tier is called the application server or the Web server, depending on the application. This server plays an intermediary role by running application programs and storing business rules (procedures or constraints) that are used to access data from the database server. Clients contain user interfaces and Web browsers. The intermediate server accepts requests from the client, processes the request and sends database queries and commands to the database server, and then acts as a conduit for passing (partially) processed data from the database server to the clients.

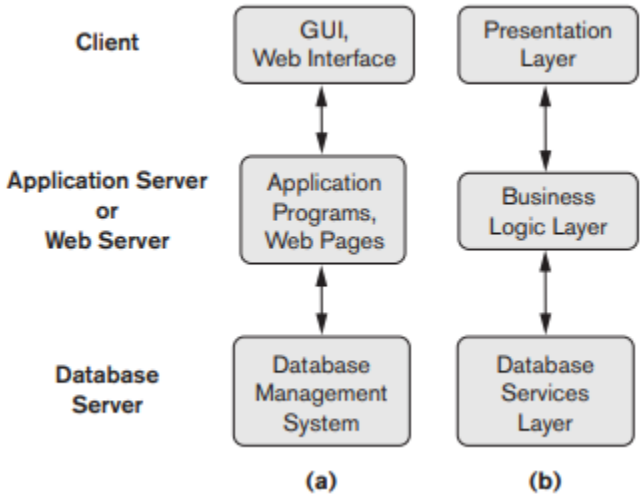


Figure 2.7
Logical three-tier client/server architecture, with a couple of commonly used nomenclatures.

Figure 2.7(b) shows another view of the three-tier architecture used by database and other application package vendors

2.6 Classification of Database Management Systems

1) According to Data Model: Main Data Model used in current commercial DBMS is RELATIONAL Data Model. Some applications use hierarchical, network, object, relational etc. Relational Database Management System uses high level query language called SQL. Object Data Model data represents in terms of object, their properties and relationship. Object with same structure and behavior belong to a class. Classes are organized into hierarchies. Operation of each class can be specified in

methods. The hierarchical model represents data as hierarchical tree structures. Each hierarchy represents a number of related records.

2) Based on number of users

Single-user systems support only one user at a time and are mostly used with PCs. **Multiuser systems**, which include the majority of DBMSs, support concurrent multiple users.

3)Based on number of sites over which the database is distributed.

A centralized DBMS if the data is stored at a single computer site.

A distributed DBMS (DDBMS) can have the actual database and DBMS software distributed over many sites connected by a computer network.

Homogeneous DDBMSs use the same DBMS software at all the sites, whereas **heterogeneous** DDBMSs can use different DBMS software at each site.

4)Based on cost

Well, it is quite difficult to classify the database on the basis of its cost as nowadays you can have free open source DBMS products such as MySQL and PostgreSQL. Although the personal version of RDBMS can cost up to $100

5) Based on Usage

The **general-purpose** DBMS is the one that is designed to meet the need of as many applications as possible.

The **special-purpose** DBMS is the one that is designed for a specific application and it can not be used for another application without performing any major changes we refer to this as online transaction processing (OLTP).

Chapter 2: E-R MODEL

The ER model describes data as entities, relationships, and attributes

3.3 Entity Types, Entity Sets, Attributes, and Keys

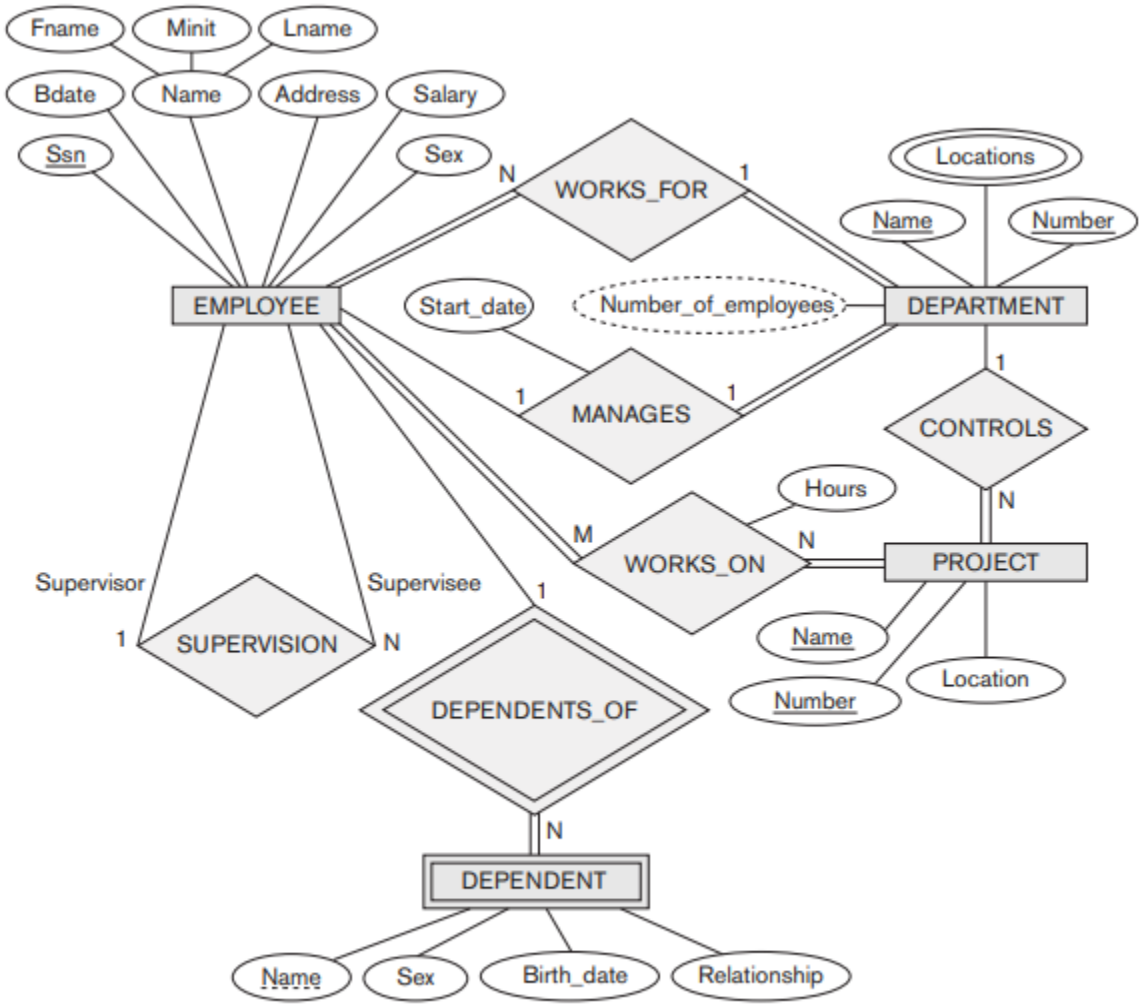


Figure 3.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter and is summarized in Figure 3.14.

Entities and Their Attributes. The basic concept that the ER model represents is an **entity**, which is a thing or object in the real world with an independent existence. It is represented using rectangle. Each entity has **attributes**—the particular properties that describe it. It is represented using oval shape.

For example, an EMPLOYEE entity may be described by the employee's name, age, address, salary, and job**.**

Several types of attributes occur in the ER model:

Composite versus Simple (Atomic) Attributes.

- Composite attributes can be divided into smaller subparts, which represent more basic attributes with independent meanings.

  For example, the Address attribute of the EMPLOYEE entity can be subdivided into Street_address, City, State, and Zip

- Simple (Atomic) Attributes: The attribute which takes up only a single value for each entity instance is a single-valued attribute.

  Example: The age of a student.

Single-Valued versus Multivalued Attributes

Most attributes have a single value for a particular entity; such attributes are called **single-valued**. For example, Age is a single-valued attribute of a person.

In some cases an attribute can have a set of values for the same entity—for instance is known as multivalued attributes. Example Colors attribute for a car.

Stored versus Derived Attributes.

The stored attribute are those attribute which doesn't require any type of further update since they are stored in the database.
Example: DOB(Date of birth) is the stored attribute.

An attribute that can be derived from other attributes is derived attributes.
Example: Total and average marks of a student
NULL Values.
In some cases, a particular entity may not have an applicable value for an attribute.

Complex attribute :

Those attributes, which can be formed by the nesting of composite and multi-valued attributes, are called "*Complex Attributes*".

3.3.2 Entity Types, Entity Sets, Keys, and Value Sets

An **entity type** defines a collection (or set) of entities that have the same attributes. Each entity type in the database is described by its name and attributes.

The collection of all entities of a particular entity type in the database at any point in time is called an **entity set or entity collection**; the entity set is usually referred to using the same name as the entity type, even though they are two separate concepts.

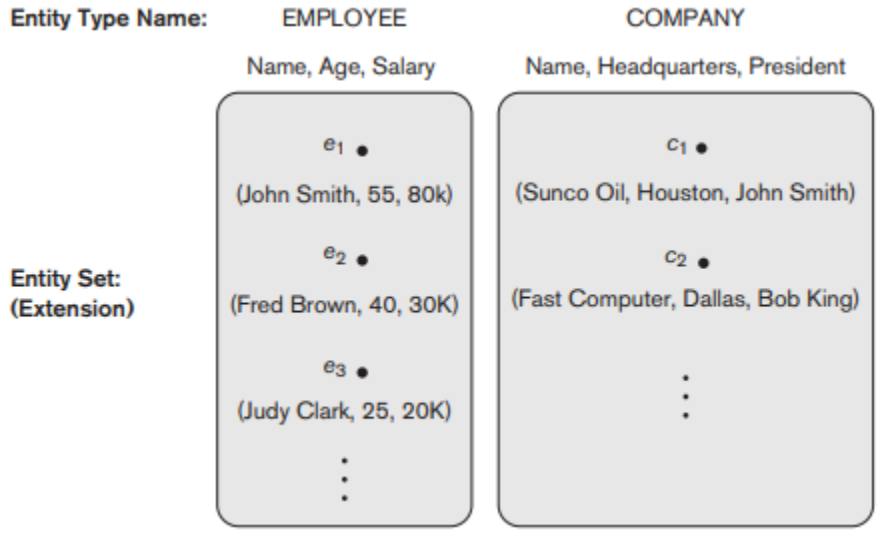
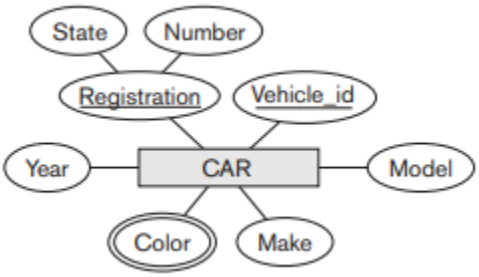


Figure 3.6
Two entity types, EMPLOYEE and COMPANY, and some member entities of each.

An entity type usually has one or more attributes whose values are distinct for each individual entity in the entity set. Such an attribute is called a **key attribute**, and its values can be used to identify each entity uniquely. For example, the ID attribute is a key of the EMPLOYEE entity type. In ER diagrammatic notation, each key attribute has its name underlined inside the oval.



Value Sets (Domains) of Attributes. Each simple attribute of an entity type is associated with a value set (or domain of values), which specifies the set of values that may be assigned to that attribute for each individual entity. In Figure 3.6, if the range of ages allowed for employees is between 16 and 70, we can specify the value set of the Age attribute of EMPLOYEE to be the set of integer numbers between 16 and 70

3.4 Relationship Types, Relationship Sets, Roles, and Structural Constraints

The attribute Department of EMPLOYEE refers to the department for which the employee works; and so on. In the ER model, these references should not be represented as attributes but as **relationships**.

3.4.1 Relationship Types, Sets, and Instances

A **relationship type** R among n entity types E1, E2, . . . , En defines a set of associations—or a **relationship set**—among entities from these entity types.

A relationship type and its corresponding relationship set are customarily referred to by the same name, R.

Mathematically, the relationship set R is a set of relationship instances r_i, where each r_i associates n individual entities (e_1, e_2, . . . , e_n), and each entity ej in ri is a member of entity set E_j , $1 \leq j \leq n$

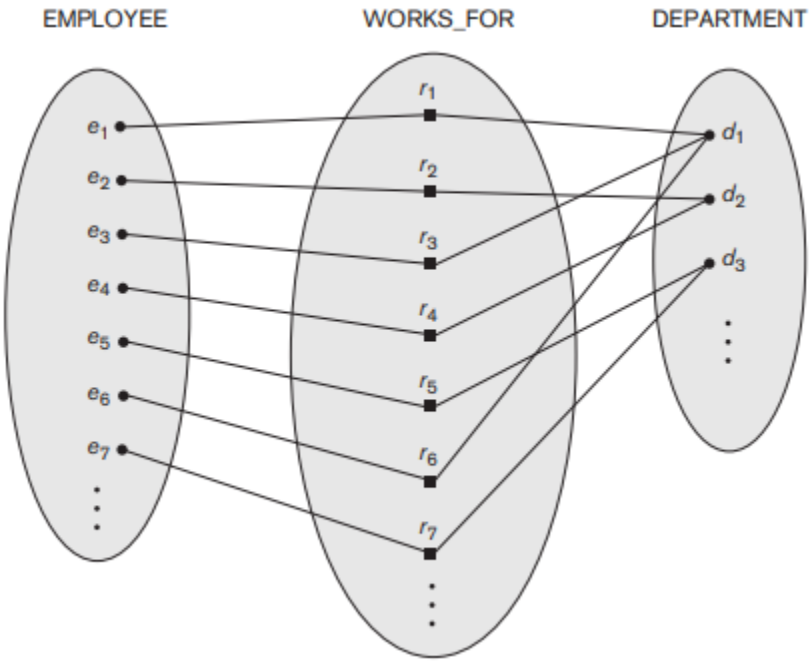


Figure 3.9
Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

3.4.2 Relationship Degree, Role Names, and Recursive Relationships

Degree of a Relationship Type. The degree of a relationship type is the number of participating entity types. Hence, the WORKS_FOR relationship is of degree two.

A relationship type of degree two is called binary, and one of degree three is called ternary

The role name signifies the role that a participating entity from the entity type plays in each relationship instance, and it helps to explain what the relationship means. For

example, in the WORKS_FOR relationship type, EMPLOYEE plays the role of employee or worker and DEPARTMENT plays the role of department or employer.

In some cases the same entity type participates more than once in a relationship type in different roles. In such cases the role name becomes essential for distinguishing the meaning of the role that each participating entity plays. Such relationship types are called recursive relationships or self-referencing relationships. Figure 3.11 shows an example. The SUPERVISION relationship type relates an employee to a supervisor, where both employee and supervisor entities are members of the same EMPLOYEE entity set. Hence, the EMPLOYEE entity type participates twice in SUPERVISION: once in the role of supervisor (or boss), and once in the role of supervisee (or subordinate). Each relationship instance ri in SUPERVISION associates two different employee entities ej and ek, one of which plays the role of supervisor and the other the role of supervisee. In Figure 3.11, the lines marked ‘1’ represent the supervisor role, and those marked ‘2’ represent the supervisee role; hence, e1 supervises e2 and e3, e4 supervises e6 and e7, and e5 supervises e1 and e4. In this example, each relationship instance must be connected with two lines, one marked with ‘1’ (supervisor) and the other with ‘2’ (supervisee).

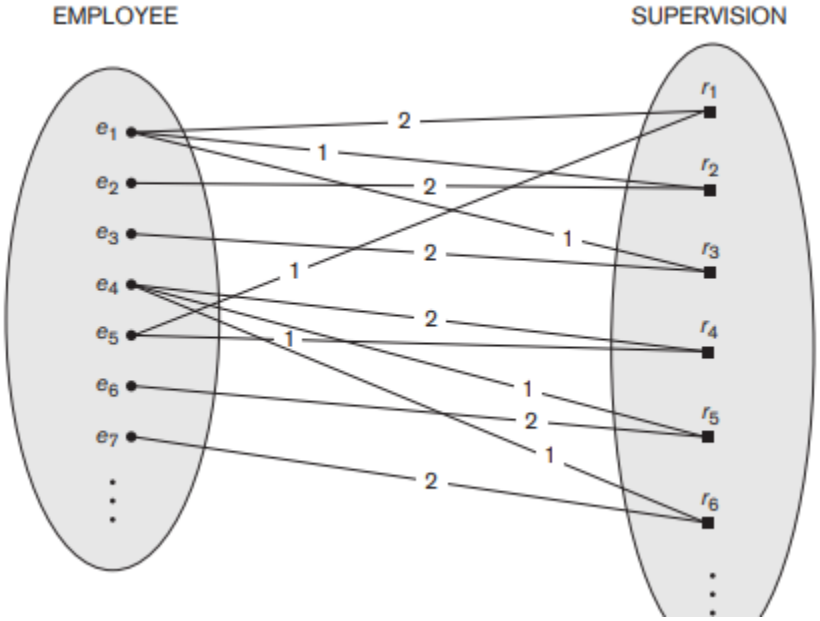


Figure 3.11
A recursive relationship SUPERVISION between EMPLOYEE in the *supervisor* role (1) and EMPLOYEE in the *subordinate* role (2).

3.4.3 Constraints on Binary Relationship Types

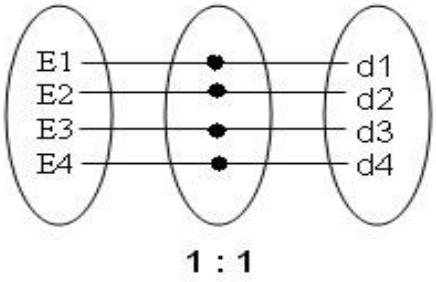
Relationship types usually have certain constraints that limit the possible combinations of entities that may participate in the corresponding relationship set.

We can distinguish two main types of binary relationship constraints: cardinality ratio and participation.

- **Cardinality Ratios for Binary Relationships.** The cardinality ratio for a binary relationship specifies the maximum number of relationship instances that an entity can participate in. for example in the works for binary relationship type, **department : employee** is of cardinality ratio 1:N, meaning that each department can be related to any number of employees but an employee can be related to only one department. The possible cardinality ratios for binary relationship types are 1:1, 1:N, N:1, and M:N.

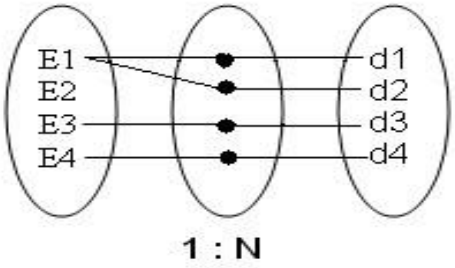
One to One

Entity types that do not have any key attribute of their own are called week entity type. Week entity type is represented in ER diagram by double rectangle like



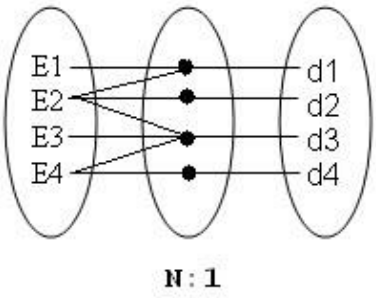
One to Many

An entity in entity type (A) can be associated with any number of entities in another entity type (B), and an entity in entity type (B) can be associated with any number of entities in entity type (A).



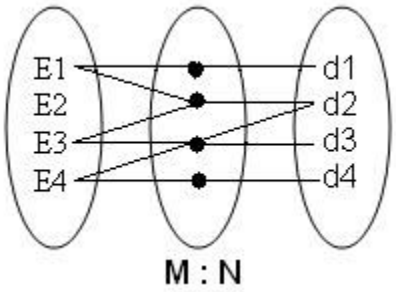
Many to One

An entity in an entity type (A) is associated with an entity in another entity type (B), and an entity in entity type (B) can be associated with any number of entities in entity type (A).



Many to Many

An entity in entity type (A) can be associated with any number of entities in another entity type (B), and an entity in entity type (B) can be associated with any number of entities in entity type (A).



Participation Constraints and Existence Dependencies.

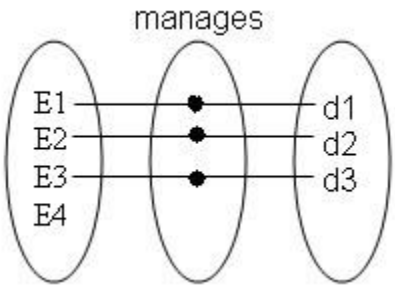
The participation constraint specifies whether the existence of an entity depends on its being related to another entity via the relationship type. This constraint specifies the minimum number of relationship instances that each entity can participate in, and is sometime called **minimum cardinality constraint**.

There are two types of participation constraints

1. **Total participation constraints**
2. **Partial participation constraints**

"The participation of an entity type E in a relationship type R is said to be **total** if every entity in entity type E participates in at least one relationship in relationship type R". For example if a company policy states that every **employee** must work for a department, then an employee entity can exist only if it participate in at least one works for relationship instance. Thus the participation of **employee** in **works for** relationship type is called **total participation**. **Total participation** is also called **existence dependency**.

In some cases we do not expect every employee to manage a department, so the participation of employee in the "**manages**" relationship type is **partial**, meaning that some or "**part of the set of**" employee entities are related to some department entity via **manages**, but not necessarily all. "If only some entities in entity type E participate in relationship in relationship type R, the participation of entity type E in relationship type R is said to be **partial**"



3.5 Weak Entity Types

Entity types that do not have any key attribute of their own are called week entity type. Week entity type is represented in ER diagram by double rectangle. Weak entity is **depend on strong entity** to ensure the existence of weak entity

3.6 ER Diagrams, Naming Conventions, and Design Issues

ER Diagram stands for Entity Relationship Diagram, also known as ERD is a diagram that displays the relationship of entity sets stored in a database. In other words, ER diagrams help to explain the logical structure of databases. ER diagrams are created based on three basic concepts: entities, attributes and relationships.

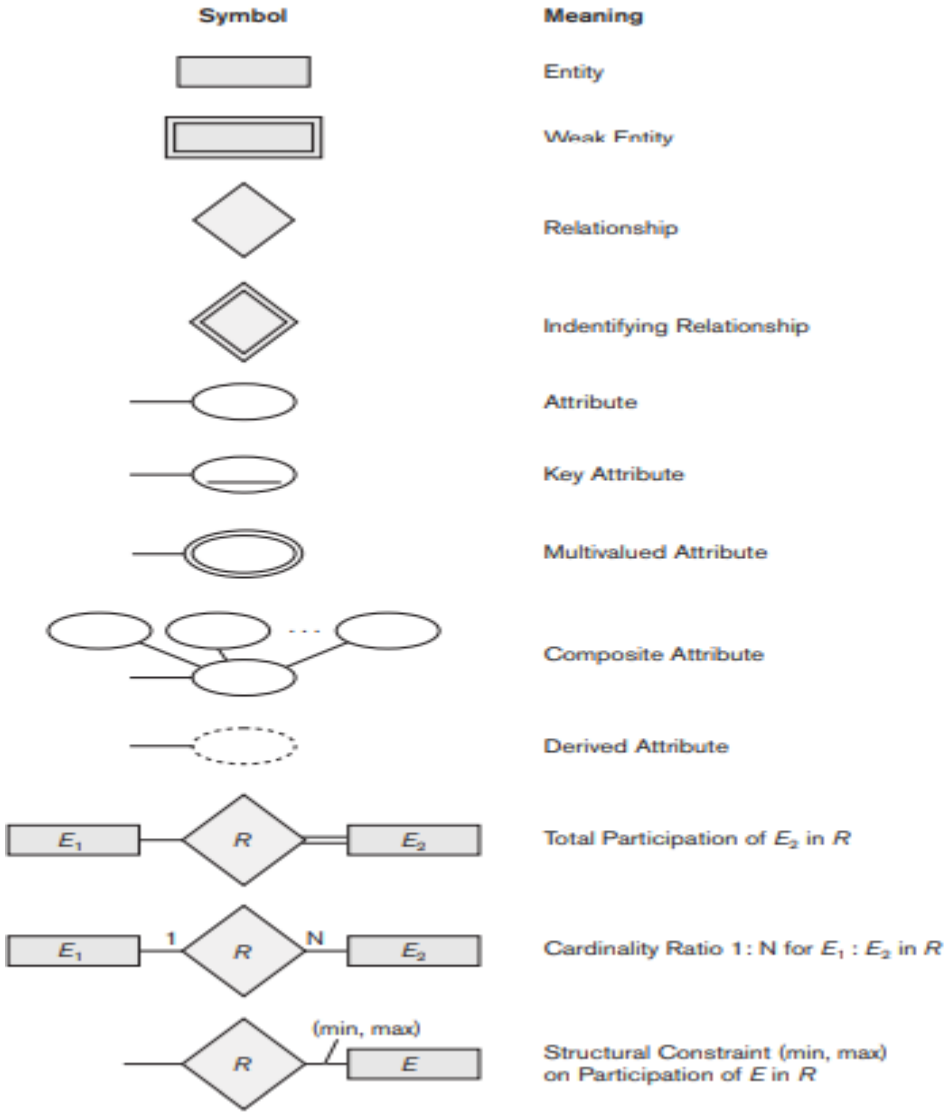


Figure 3.14 Summary of the notation for ER diagrams.

How to Create an Entity Relationship Diagram (ERD)

Step 1) Entity Identification

Step 2) Relationship Identification

Step 3) Cardinality Identification

Step 4) Identify Attributes

Step 5) Create the ERD Diagram

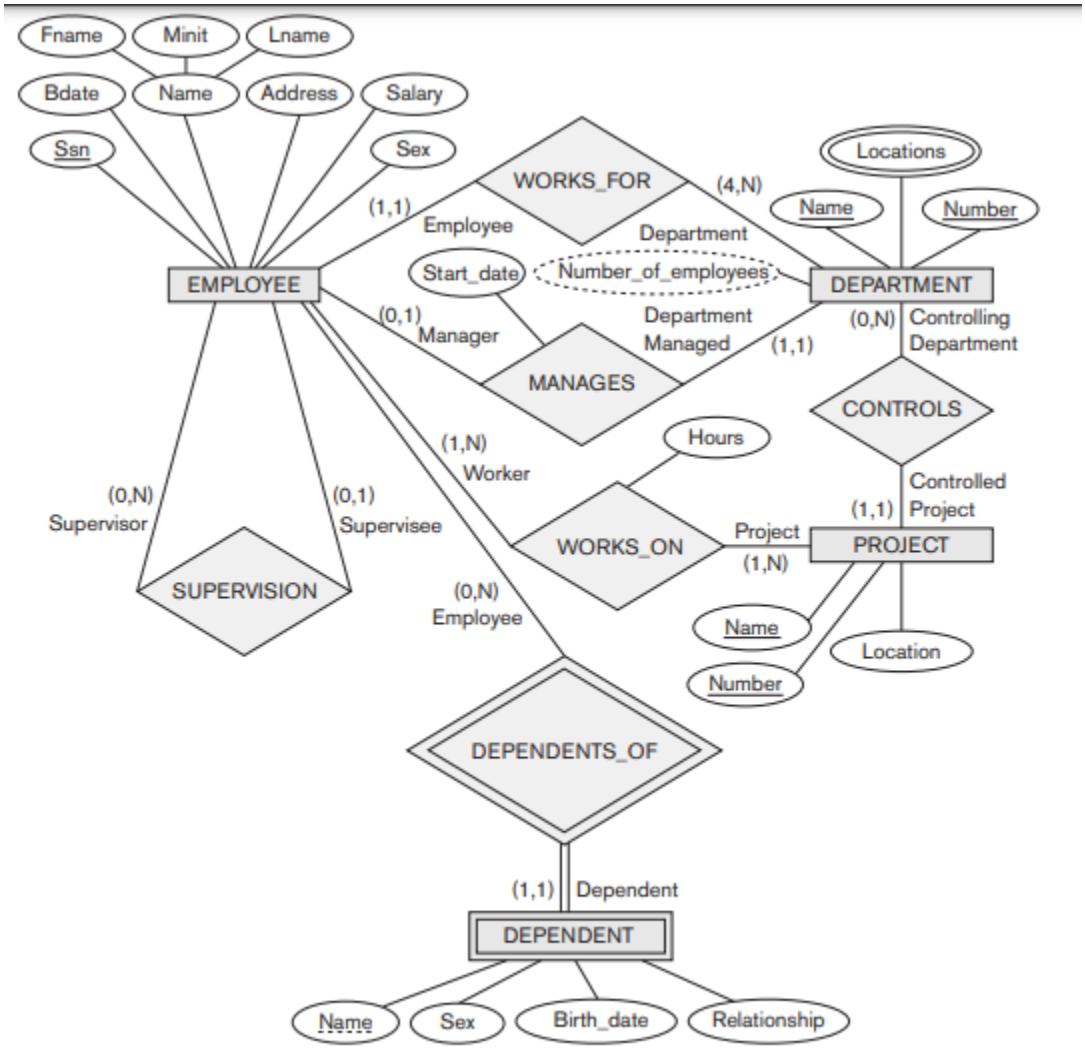


Figure 3.15
ER diagrams for the company schema, with structural constraints specified using (min, max) notation and role names.